# C_Beyond_Gravity

April 21, 2026

# 1 Project 1: Time-frequency analysis - Part C: Time-frequency analysis

:::{important} Learning Goals - [ ] I can determine the frequency content in different time windows of a signal. - [ ] I understand the effect of window length on the resolution of a spectral estimate. - [ ] I am able to find characteristic spectral peaks in a signal. ::: _____

## 1.1 Table of Contents

### 1.1.1

### 1.1.2

### 1.1.3

---

(h-tf_content)= # Exercise 1: Time-dependent frequency content

::::{tip} Exercise 1: Download and study this test dataset from Beyond Gravity

```
Open a Terminal with Menu 'File' -> 'New' -> 'Terminal' and run the following line

wget https://polybox.ethz.ch/index.php/s/PYLxFNwEojJBgKC/download -O data/yLLS_1.txt

Try to load the file the same way as before
```

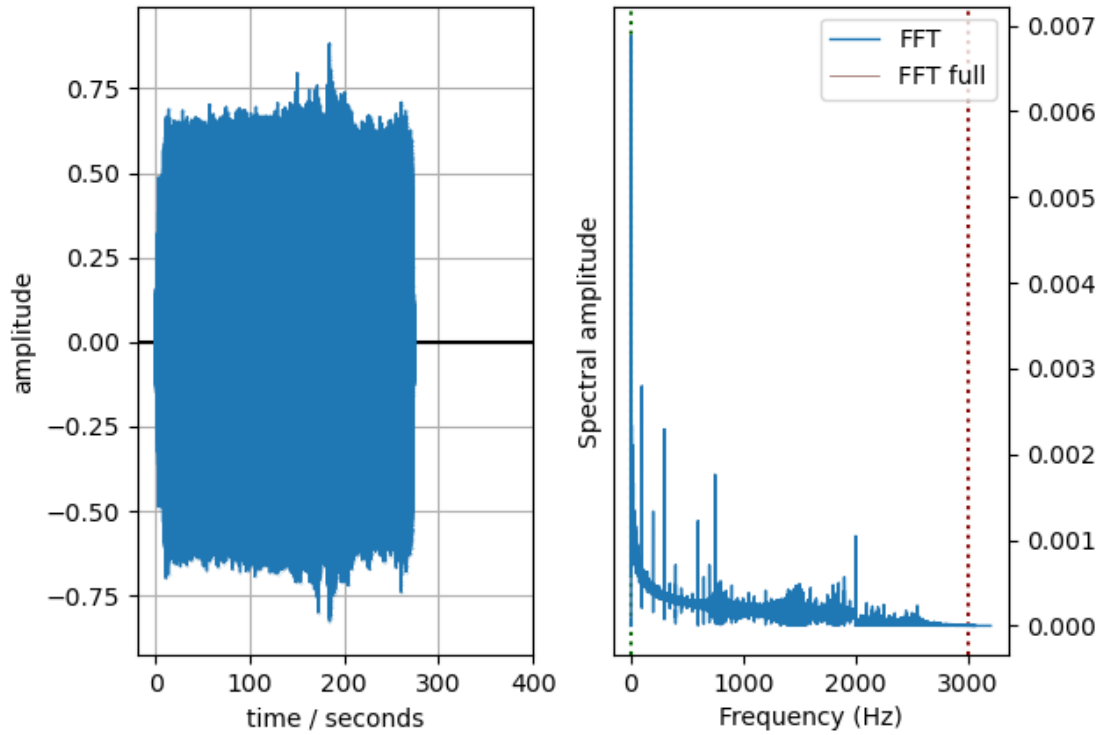
::::

Next, use the code below to plot time series and spectrum of the signal. You can restrict the time range and spectral range and look at the effect on the other window.

[5]:
```
import matplotlib.pyplot as plt
import ipywidgets as widgets
from functions.aliasing import open_figures, filtererer
import numpy as np
import pandas as pd
%matplotlib widget

trace = 1

# Modify this line to match the file name
```

```
df = pd.read_csv('data/yLLS_1.txt', encoding='latin1', sep=' ',
                 skipinitialspace=True, skiprows=46,
                 usecols=((trace - 1) * 2, (trace - 1) * 2 + 1))


x = np.asarray(df.iloc[:,0])
y = np.asarray(df.iloc[:,1])

plt.close('all')

# set up plot to speed up execution time later
fig, ax = plt.subplots(nrows=1, ncols=2) # , figsize=(10, 4))
ax_time = ax[0]
ax_freq = ax[1]
ax_time.grid(True)

ax_time.set_xlabel('Time (s)')
ax_freq.set_xlabel('Frequency (Hz)')
# ax_freq.set_xlim(0, 10.)
ax_time.set_ylabel('Amplitude')
ax_freq.set_ylabel('Spectral amplitude')

ax_time.plot(x, y)
ax_time.set_xlabel('time / seconds')
ax_time.set_ylabel('amplitude')
ax_freq.yaxis.tick_right()

n_times = len(y)
dt = x[1] - x[0]

_ = widgets.interact(filtererer,
                     ax_freq=widgets.fixed(ax_freq),
                     ax_time=widgets.fixed(ax_time),
                     data=widgets.fixed(y),
                     dt=widgets.fixed(dt),
                     freq_highpass=(1, 3000, 10),
                     freq_lowpass=(1, 3000, 10),
                     t_start=(0., 400.),
                     t_end=(0., 400.),
                     norm=['forward', 'backward', 'ortho']
                    )
plt.show()
```

```
interactive(children=(FloatSlider(value=0.0, description='t_start', max=400.0),␣
↪FloatSlider(value=400.0, descr…
```




### 1.2 Exercise 2: What kind of signal are we looking at?

Run some tests on the signal and describe what you think the signal character is like (5 minutes). 1. Restrict the time range and look at the spectrum 2. Restrict the spectral range and look at the time series

[3]:
```
from IPython.display import IFrame
IFrame('https://speakup.epfl.ch/room/join/60909', 600, 600)
```

[3]:
```
<IPython.lib.display.IFrame at 0x7fc92e5e8590>
```

---

(h-spectrogram)= # Exercise 2: The spectrogram

Run the code below and look at the plot.

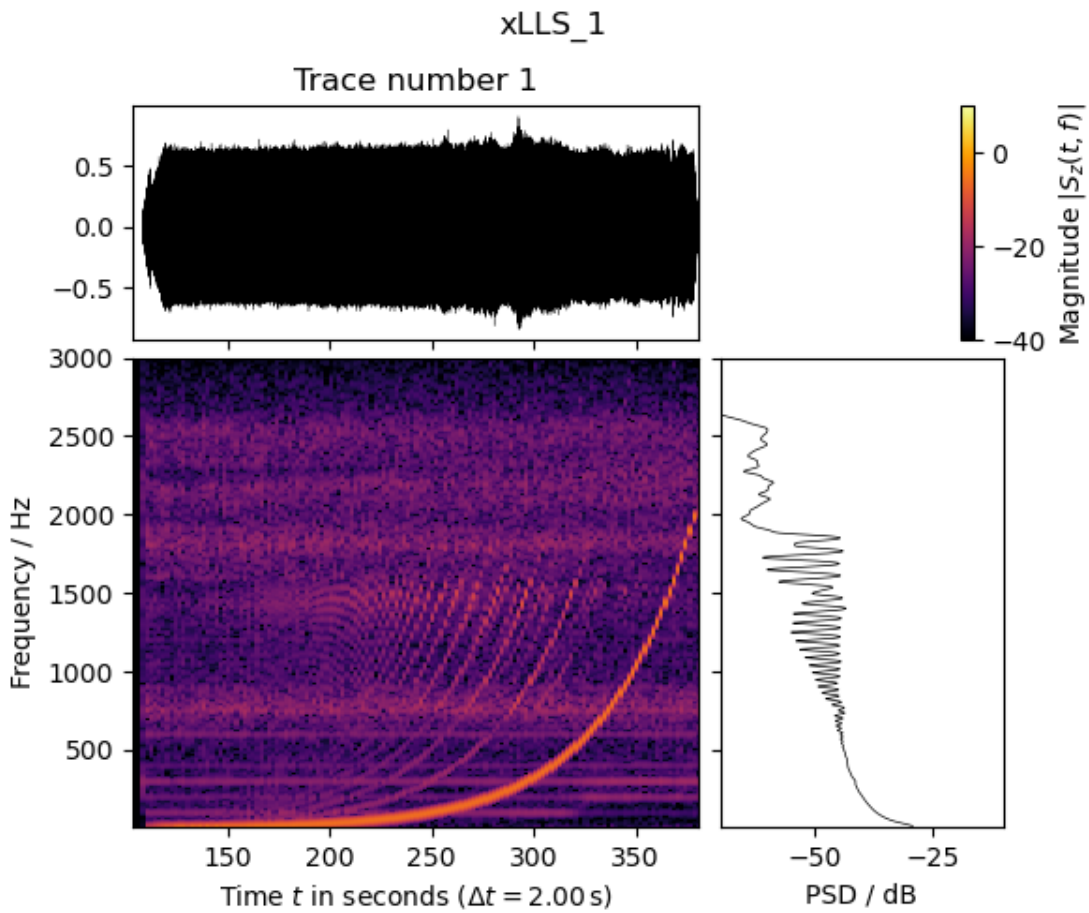
1. Discuss in groups of four what you see (10 minutes).
2. Adapt the parameters winlen_sec and overlap and describe the changes.

[8]:
```
import matplotlib.pyplot as plt
import numpy as np
from tqdm import tqdm
```

```python
from scipy.fft import fftshift
from scipy.signal import stft, istft, spectrogram, ShortTimeFFT, welch

# Close any remaining plots from previous cells to save memory. This is␣
↪generally good practice in Jupyter Notebooks
plt.close()

# Choose window length for spectrogram.
winlen_sec = 10

# Choose overlap ratio between time windows
overlap = 0.8

# We can re-use the trace from the previous cell
# Load data trace
experiment = 'LLS_1'
comp = 'x'
trace = 1
'''
df = pd.read_csv(f'data/Data_BG/{comp}{experiment}.txt',
                 encoding='latin1', sep=' ',
                 skipinitialspace=True, skiprows=46,
                 usecols=((trace - 1) * 2, (trace - 1) * 2 + 1))

# Create the data vectors and determine their dimensions
x = np.asarray(df.iloc[:,0])
y = np.asarray(df.iloc[:,1])
'''
n_times = len(y)
dt = x[1] - x[0]

# Get sampling frequency and number of samples
fs = 1./dt

# Set parameters of spectrogram based on window length chosen on top
nperseg = int(winlen_sec * fs)
noverlap = int(nperseg * overlap)
vmin = -40
vmax = 10


# Define STFT object:
win = ('gaussian', 1e-2 * fs)  # Gaussian with 0.01 s standard dev.
SFT = ShortTimeFFT.from_window(win, fs, nperseg, noverlap, fft_mode='centered',
                               scale_to='magnitude', phase_shift=None)
# Compute STFT
Sz1 = SFT.stft(y)
```

```
# Create plot figure with 4 subplots:
gridspec_kw = dict(width_ratios=[2,1], height_ratios=[1,2], wspace=0.05,␣
↪hspace=0.05)
fig1, axx = plt.subplots(2, 2, sharex='col', sharey='row',
                         figsize=(6., 5.),
                         gridspec_kw=gridspec_kw)  # enlarge figure a bit
ax_time = axx[0][0]
ax_specgram = axx[1][0]
ax_spectrum = axx[1][1]
ax_cb = axx[0][1]

# Define extend of plot windows
t_lo, t_hi, f_lo, f_hi = SFT.extent(n_times, center_bins=True)
t_lo += x[0]
t_hi += x[0]

ax_time.set_xlim(t_lo, t_hi)
ax_specgram.set_ylim(0, f_hi)

ax_time.set_title(f"Trace number {trace}")
ax_specgram.set_xlabel(rf"Time $t$ in seconds ($\Delta t= {SFT.delta_t:0.
↪2f}\,$s)")
ax_specgram.set_ylabel(r"Amplitude / AU")
ax_specgram.set_ylabel(r"Frequency / Hz")

# Plot time series in top left subplot
ax_time.plot(x, y, lw=0.2, c='k')

# Compute spectral density function using Welch's method with same parameters␣
↪as spectrogram.
f, pxx = welch(x=y,
               fs=fs,
               window=win,
               nperseg=nperseg,
               noverlap=noverlap)

# Plot spectrum in bottom right subplot
ax_spectrum.plot(10*np.log10(pxx), f, 'k', lw=0.5)
ax_spectrum.set_xlim(-70, -10)

# Plot spectrogram in bottom left subplot
kw = dict(origin='lower', aspect='auto', cmap='inferno',
          vmin=vmin, vmax=vmax, interpolation='nearest')
im1b = ax_specgram.imshow(10*np.log10(abs(Sz1) + 1e-10),
                          extent=(t_lo, t_hi, f_lo, f_hi), **kw)
```

```
# Create colorbar for spectrogram
ax_cb.set_axis_off()
fig1.colorbar(mappable=im1b, ax=ax_cb, label="Magnitude $|S_z(t, f)|$")
ax_spectrum.set_xlabel('PSD / dB')

fig1.suptitle(f'{comp}{experiment}')

# ax_specgram.set_yscale('log')
ax_specgram.set_ylim(10, 3000)
plt.show()
```




---

(h-hw)= # Homework: Correct amplitudes

Here is a description of the dataset from BG. The whole dataset is available at

[https://polybox.ethz.ch/index.php/s/TLCRBYL0jiVSD3E]

For each of the three axis, the following tests were performed:

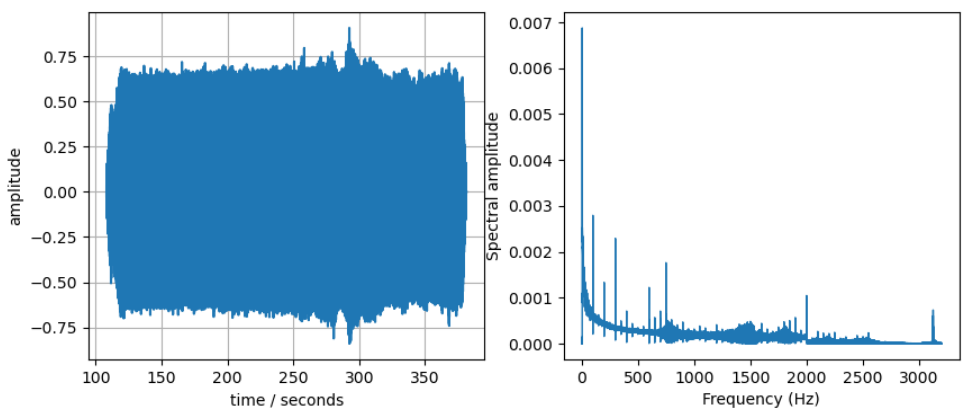
- LLS_1 -> first resonance search, 5-2000 Hz, 0.5g
- RND_1 -> random vibration test for a duration of 1 minute
- LLS_2 -> second resonance search, same parameters as first one

The Dataset contains signals for 2 control sensors (which are used for the control feedback loop of the shaker) and 5 measurement sensors, positions marked in the attached pictures. All sensors are triaxial sensors.

The text files contain raw sensor data in the time domain. Each sensor channel (3 axes/channels per sensor) is represented as two columns (time and acceleration) with a header containing info about channel name and direction.

`{image} imgs/BG_sample.png :alt: Rendering of test sample :width: 300px  {image} imgs/BG_schnitt.png :alt: Cross section of test sample :width: 300px` ::::{tip} *Exercise H1:* Analysis of test dataset Work your way through the headers and try to 1. Figure out the physical quantities of the measurement. 2. Figure out the correction factor to arrive at proper physical quantities and apply to the code from Exercise 1 or the one below: 3. Create a plot of the time series and the power spectral density with correct physical quantities. 4. List at least three resonance frequencies of the sample. 5. Compare resonance frequencies before the random vibration test (LLS_1) and after (LLS_2) ::::

[5]:
```
import matplotlib.pyplot as plt
from functions.aliasing import open_figures
import numpy as np
%matplotlib widget
x = df.iloc[:,0]
y = df.iloc[:,1]

plt.close('all')
# set up plot to speed up execution time later
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
ax_time = ax[0]
ax_freq = ax[1]
# ax_time.set_ylim([-4, 4])
ax_time.grid(True)

ax_time.set_xlabel('Time (s)')
ax_freq.set_xlabel('Frequency (Hz)')
# ax_freq.set_xlim(0, 10.)
ax_time.set_ylabel('Amplitude')
ax_freq.set_ylabel('Spectral amplitude')

ax_time.plot(x, y)
ax_time.set_xlabel('time / seconds')
ax_time.set_ylabel('amplitude')
print(x.shape, y.shape)
n_times = len(y)
print(n_times)
dt = x[1] - x[0]
```

```python
print(dt)
# Compute and plot spectrum with FFT
spec_fft = abs(np.fft.fft(y, norm='forward'))  # / n_times

freq_fft = np.fft.fftfreq(n=n_times, d=dt)
# freq_fft_full = np.fft.fftfreq(n=n_times_full, d=dt_full)

ax_freq.plot(freq_fft[0:n_times // 2],
             spec_fft[0:n_times // 2],
             color='C0', lw=1, label="FFT", zorder=10)
plt.show()
```

```
(1748991,) (1748991,)
1748991
0.0001563000000004422
```